

Obsah

1	MVC architektura.....	2
1.1	Princip MVC.....	2
1.2	Cíl.....	3
1.3	Komponenty.....	4
1.4	Model.....	4
1.5	Pohled.....	4
1.6	Kontroler.....	5
1.7	Životní cyklus stránky.....	6
1.8	Vlastní realizace první aplikace.....	7
1.8.1	.htaccess.....	7
1.9	Poznámka k .htaccess.....	7
1.10	Mod_rewrite.....	7
1.11	Dá se mod_rewrite použít?.....	7
1.12	Příklad jednoduchého přesměrování.....	8
1.12.1	Příklad jednoduchého podstrčení obsahu.....	8
1.13	Obecný zápis RewriteRule.....	9
1.14	Podstrkávání a přesměrování.....	9
1.14.1	Podstrkávání.....	9
1.14.2	Přesměrování místo podstrčení.....	10
1.14.3	Kdy se podstrkává a kdy přesměrovává.....	10
1.15	Příklad jednoduchého regulárního výrazu.....	10
1.16	Proměnné z regulárů.....	10
1.17	Podmínky RewriteCond.....	11
1.17.1	Přesměrování na doménu s www.....	11
1.17.2	Přesměrování na doménu bez www.....	12
1.17.3	Podobný příklad: přesměrování staré domény na novou.....	12
1.17.4	Symbolický zápis RewriteCond.....	12
1.18	Příznaky v hranatých závorkách.....	12
1.18.1	Nastavení .htaccess.....	13
2	Composer.....	13
2.1	Soubory Composeru.....	13
2.2	Příkazy z příkazové řádky.....	14
3	ORM.....	14
4	Doctrine (PHP).....	15

4.1	Struktura frameworku.....	15
4.2	Požadavky.....	16
4.3	Definice a práce entitami.....	16
5	DQL.....	18
6	Framework.....	19
6.1	Architektura frameworku.....	19
6.2	Příklady Frameworků.....	19
6.3	Laravel.....	19
6.4	CodeIgniter.....	20
6.5	Symfony.....	20
6.6	CakePHP.....	21
6.7	Yii.....	22
6.8	Zend Framework.....	22
6.8.1	Důvody pro použití Zendu.....	22
6.9	Phalcon.....	23
6.9.1	Důvody použití Phalconu.....	23
6.10	Swift.....	23
6.10.1	Důvody použití Swift.....	23
6.11	PHPixie.....	24
6.11.1	Důvody použití PHPixie.....	24
6.12	10. Slim.....	24
6.13	Nette.....	25
7	GitHub.....	25
7.1	Možnosti využití.....	25

1 MVC architektura

MVC je velmi architektonický vzor, který se uchytil zejména na webu, ačkoli původně vznikl na desktopech.

Je součástí populárních webových frameworků, jakými jsou např. Zend nebo Nette pro PHP, Ruby On Rail pro Ruby nebo MVC pro ASP .NET.

1.1 Princip MVC

Základní myšlenkou MVC architektury

- Oddělení logiky od výstupu.

- Řeší tedy problém tzv. "špagetového kódu", kdy máme v jednom souboru (třídě) **logické operace** a zároveň **renderování výstupu**.
- Soubor obsahuje databázové dotazy, logiku (např. PHP operace) a HTML tagy.

```
$vysledek=mysql_query("select * from otazky", $spojeni);
if (mysql_num_rows($vysledek)==0)
    echo '<p>Zatím tu žádnou anketu nemáme.</p>';
else
{
    echo '<ul>';
    while ($radek = mysql_fetch_array($vysledek))
    if ($vybranaanketa == $radek['kodankety'])
        echo '<li>' . $radek['kodankety'] . ': ' . $radek['otazka'] . ' (aktuálně vybraná)</li>';
    else
        echo '<li><a href=tvorbaanket.php?vybranaanketa=' . $radek['kodankety'] . "'>'
        . $radek['kodankety'] . ': '
        . $radek['otazka'] . '</a></li>';
    echo '</ul>';
}
```

Bez MVC

- Kód se špatně udržuje a rozšiřuje.
- HTML není správně naformátováno, ztrácíme se v jeho stromové struktuře.

1.2 Cíl

- zdrojový kód s logikou má vypadat jako zdrojový kód (např. PHP)
- výstup vypadal jako HTML stránka s co nejmenší příměsí dalšího kódu.
- kód výše by se tedy rozdělil do 2 souborů (HTML šablony a PHP skriptu), ideálně ještě s použitím dalších tříd. Již brzy v seriálu uvidíme, jak kód MVC aplikace vypadá.

1.3 Komponenty

Celá aplikace MVC je rozdělena na **komponenty** 3 typů:

1. **Modely**
2. **View (pohledech)**
3. **Controllerech (kontrolerech),**

od toho MVC.

Komponenty jsou samozřejmě **třídy**, které mohou být děděné z abstraktních předků, jen view je většinou jen jako HTML šablona. Pojďme si jednotlivé komponenty nejprve popsat.

1.4 Model



- Model obsahuje logiku a vše, co do ní spadá.
- Mohou to být výpočty, databázové dotazy, validace a podobně.
- Model vůbec neví o výstupu.
- Jeho funkce spočívá v přijetí parametrů zvenku a vydání dat ven. Zdůrazním, že parametry nemyslím URL adresu ani žádné jiné parametry od uživatele.
- Model neví, odkud data v parametrech přišla a ani jak budou výstupní data zformátována a vypsána.

Využívá se principu manažerů, logika aplikace tedy bude rozdělena např. mezi manažer: ManazerUzivatele, ManazerClanku a tak podobně.

1.5 Pohled



Pohled (View) se stará o zobrazení výstupu uživateli.

Nejčastěji se jedná o phtml šablonu, obsahující HTML stránku a tagy nějakého značkovacího jazyka, který umožňuje do šablony vkládat proměnné, případně provádět iterace (cykly) a podmínky.

Pro šablony se často používají speciální značkovací jazyky (např. Smarty).

Pohledů máme mnoho, např. pro funkcionalitu s entitou uživatele: *uzivatel_registrace*, *uzivatel_prihlaseni*, *uzivatel_profil* a podobně. Pohled *uzivatel_profil* je ale již společný všem uživatelům a jsou do něj posílána různá data, vždy podle toho, koho zrovna zobrazujeme. Tato data jsou poté dosazena do HTML elementů šablony.

Pohledy lze samozřejmě vkládat do sebe, abychom se neopakovali (šablona s rozložením stránky, šablona s menu a šablona článkem).

View není jen šablona, ale zobrazovač výstupu. Obsahuje tedy minimální množství logiky, která je pro výpis nutná (např. kontrola, zda si uživatel vyplnil přezdívkou před jejím vypsáním nebo cyklus s komentáři, které se vypisují).

View podobně jako Model vůbec neví, odkud mu data přišla, stará se jen o jejich zobrazení uživateli.

1.6 Kontroler



Jedná se o jakéhosi prostředníka, se kterým komunikuje uživatel, model i pohled.

Drží tedy celý systém pohromadě a komponenty propojuje.

Opět existuje mnoho různých přístupů, nejčastěji má každá entita jeden kontroler, máme tedy *UzivatelKontroler*, *ClanekKontroler* a tak podobně.

1.7 Životní cyklus stránky

Životní cyklus zahajuje uživatel, který zadá do prohlížeče adresu webu a parametry, kterými nám sdělí, kterou podstránku si přeje zobrazit.

Budeme chtít zobrazit detail uživatele s id 15. Udělejme si ukázkou URL adresy:

`http://www.domena.cz/uzivatel/detail/15`

Požadavek jako první zachytí tzv. směrovač (router). Ten podle parametrů pozná, který kontroler voláme.

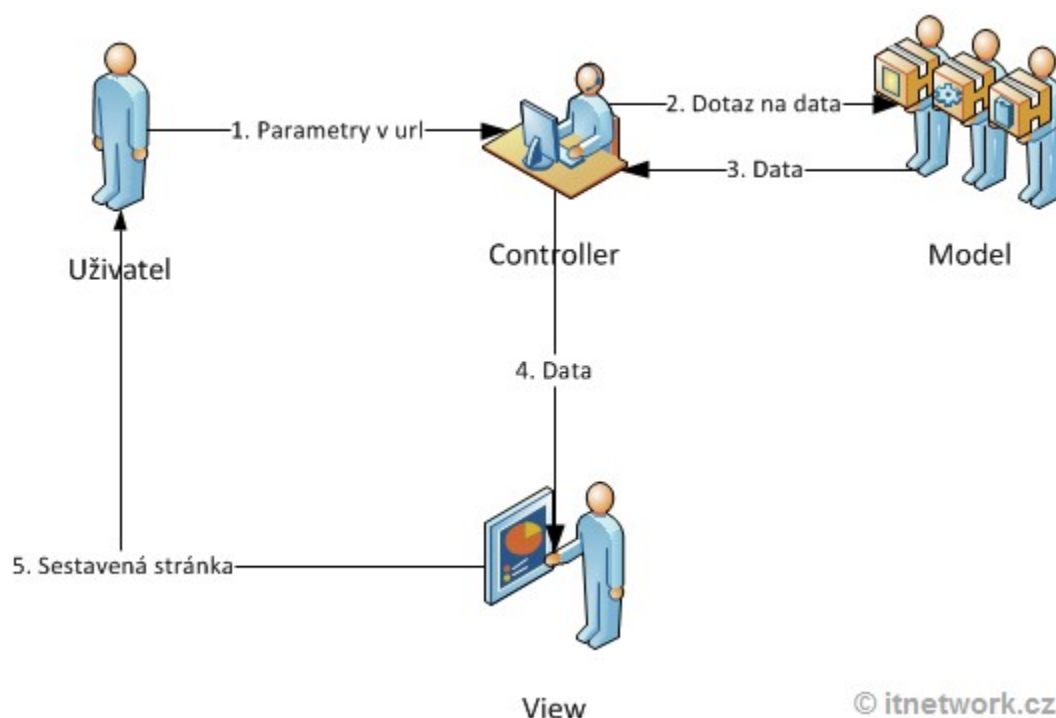
Udržujme maximální jednoduchost a vypuštěme tzv. manuální routování, název kontroleru jednoduše uvedeme jako 1. parametr v URL.

Zde je tedy zavolán kontroler `UzivatelKontroler`, kterému jsou předány 2 parametry "detail" a 15.

Daný kontroler podle parametrů pozná, co se po něm chce, tedy že má zobrazit detail uživatele. Zavolá model, který uživatele vyhledá v databázi a vrátí jeho údaje. Dále zavolá další metodu modelu, která např. vypočítá věk uživatele. Tyto údaje si kontroler ukládá do proměnných. Nakonec vypíše pohled (view). Název pohledu poznáme podle akce, kterou provádíme. Pohledu jsou předány proměnné s příslušnými daty. Kontroler tedy poslechl uživatele, obstaral podle parametrů dotazu data od modelu a předal je pohledu.

Pohled přijme data od kontroleru a vloží je do připravené šablony. Hotová stránka je zobrazena uživateli, který často o celé této kráse ani netuší

Celou situaci můžeme znázornit diagramem:



Získali jsme tedy oddělení logiky od výstupu, view jsou jako HTML, modely zas v našem skriptovacím jazyce (např. v PHP). Dosáhli jsme přehlednosti kódu, který je logicky rozčleněný.

MVC architektura nám usnadňuje i myšlení při vývoji projektu.

1.8 Vlastní realizace první aplikace

1.8.1 .htaccess

Htaccess slouží ke konfiguraci webserveru Apache a určuje, co se má stát, když uživatel napíše URL adresu.

Vše se děje ještě předtím, než se vůbec spustí jazyk PHP jako takový.

Vytvořme tedy tento soubor (i s tou tečkou na začátku názvu) v kořenové složce.

1.9 Poznámka k .htaccess

Zdroj: <http://www.jakpsatweb.cz/server/mod-rewrite.html>

1.10 Mod_rewrite

Na serveru Apache může být nainstalován modul mod_rewrite, který umožňuje nastavovat **URL adresy, jejich přesměrování a podstrkávání**.

1.11 Dá se mod_rewrite použít?

Ne na každém serveru se dá pracovat s mod_rewrite. Požadavky nutné:

- na serveru musí běžet Apache
- na tom Apači musí být mod_rewrite nainstalován a povolen
- musíte mít možnost konfigurovat server -- buďto souborem httpd.conf, nebo svým souborem .htaccess

I tak to pořád nejsou podmínky dostačující. Někdy se mohou direktivy mod_rewrite třískat s jinými existujícími direktivami.

Na mém serveru například začal mod_rewrite fungovat až poté, co jsem si do .htaccess přidal zápis

Options +FollowSymLinks

Apache Version	Apache/2.2.15 (CentOS)
Apache API Version	20051115
Server Administrator	stumbaue@student.vspj.cz
Hostname:Port	ples.vspj.cz:0
User/Group	apache(48)/48
Max Requests	Per Child: 4000 - Keep Alive: off - Max Per Connection: 100
Timeouts	Connection: 60 - Keep-Alive: 15
Virtual Server	Yes
Server Root	/etc/httpd
Loaded Modules	core prefork http_core mod_so mod_auth_basic mod_auth_digest mod_authn_file mod_authn_alias mod_authn_anon mod_authn_dbm mod_authn_default mod_authz_host mod_authz_user mod_authz_owner mod_authz_groupfile mod_authz_dbm mod_authz_default util_ldap mod_authnz_ldap mod_include mod_log_config mod_logio mod_env mod_ext_filter mod_mime_magic mod_expires mod_deflate mod_headers mod_usertrack mod_setenvif mod_mime mod_dav mod_status mod_autoindex mod_info mod_dav_fs mod_vhost_alias mod_negotiation mod_dir mod_actions mod_speling mod_userdir mod_alias mod_substitute mod_rewrite mod_proxy mod_proxy_balancer mod_proxy_ftp mod_proxy_http mod_proxy_ajp mod_proxy_connect mod_cache mod_suexec mod_disk_cache mod_cgi mod_version mod_perl mod_php5 mod_ssl mod_dav_svn mod_authz_svn mod_wsgi

1.12 Příklad jednoduchého přesměrování

Chceme, aby se stránka <http://muj-server.cz/pozadovany-soubor.html> přesměrovala na <http://muj-server.cz/vysledny-soubor.html>. Do souboru .htaccess nebo httpd.conf zadám tento kód:

```
# presmerovani
RewriteEngine on
RewriteRule pozadovany-soubor\.html /vysledny-soubor.html [R]
```

Když pak uživatel požádá o [pozadovany-soubor.html](#), tak místo něj dostane [vysledny-soubor.html](#).

Tuto novou adresu [vysledny.soubor.html](#) uvidí i v adrese prohlížeče. To [R] je jako *redirect*, tedy přesměrování (hodí ho to na jinou adresu, než chtěl).

Cíl přesměrování musí být absolutní adresa. Všimněte si, že [/vysledny.soubor.html](#) začíná lomítkem, a tak se počítá od rootu webu (a je to tedy absolutní adresa).

1.12.1 Příklad jednoduchého podstrčení obsahu

Do souboru .htaccess nebo httpd.conf se zadá tento kód:

```
# podstrceni
RewriteEngine on
RewriteRule zadana-stranka\.html podstrcena-stranka.html
```

Místo zadané stránky se pak objeví **obsah** podstrčené stránky. Zadaná **adresa** v prohlížeči ale **zůstává stejná**, totiž [zadana-stranka.html](#). Proto tomu říkám podstrčení, odborněji se tomu říká prepisování. (Adresa bude červená, obsah zelený.)

Bude se to podstrkávat, protože tentokrát tam není to [R].

Protože zápisy adres v tomto příkladu nezačínají lomítkem, adresa zadané stránky se odvozuje od adresáře, ve kterém se vyskytuje .htaccess. Stejně tak podstrcena-stranka.html (ta musela být v minulém příkladu zadávána absolutně s lomítkem na začátku). Kdyby podstrčená stránka začínala lomítkem, počítala by se nikoliv od aktuálního adresáře, ale od kořene webu.

První parametr se chápe jako regulární výraz. V zadané stránce (první parametr) se tedy musejí některým znakům předřazovat zpětná lomítka (typicky třeba před tečku), aby se tyto znaky jako regulární výraz neinterpretovaly. Proto je zadaná stránka zapsaná se zpětným lomítkem před tečkou: zadana-stranka\.html. Naopak v cílové stránce se žádné znaky nanahrazují (**druhý parametr** není regulár).

1.13 Obecný zápis RewriteRule

Zápis mod_rewrite vždy začíná jeho zapnutím:

```
RewriteEngine on
```

Je potřeba mít to tam samozřejmě vždycky, ale v příkladech to někdy vynechávám.

RewriteRule má následující syntaxi:

```
RewriteEngine on
```

```
RewriteRule Na-co-se-ptá-klient Co-skutečně-dostane [nepovinná-pravidla]
```

První řádek **RewriteEngine on** je to zaklínadlo, kterým se mod_rewrite zapíná.

RewriteRule: Na co se ptá klient (**první parametr**), se zpracovává jako cesta k souboru odvozená od rootu webu. Příklady: /index.html, /archiv/akce.php Je to regulární výraz (některé znaky se musejí escapovat). Když server vidí, že klient chce stránku, která odpovídá **prvnímu parametru**, začne něco podnikat.

Adresa stránky, kterou uživatel skutečně dostane (**druhý parametr**), je buďto absolutní adresa (začíná na http://), nebo relativní. Relativní adresa se odvozuje buďto od aktuálního adresáře, nebo od rootu webu -- to pokud zápis **Co-skutečně-dostane** začíná lomítkem. Například http://priklad.cz/soubor.html je adresa absolutní, zápis /soubor.html je adresa relativní.

Tento **druhý parametr** se narodil od prvního nezapíše jako regulární výraz, například se tedy nemusejí escapovat tečky zpětným lomítkem (ale zase když se zaescapují, tak to nevadí).

1.14 Podstrkávání a přesměrování

```
RewriteEngine on
```

```
RewriteRule zadana-stranka\.html podstrcena-stranka.html
```

V tomto příkladu se neprovede přesměrování, nýbrž **podstrčení** (není tam to **[R]**). To znamená, že uživatel stále uvidí adresu, kterou zadal (případně na kterou kliknul). Server mu na tuto adresu "podstrčí" obsah souboru **podstrcena-stranka.html**. Všimněte si, že tentokrát nejsou v **žádných hranatých závorkách** žádné kódy -- podstrkávání je výchozí chování mod_rewrite.

1.14.1 Podstrkávání

Když budu mít soubor .htaccess s **příkazem na podstrkávání** například v adresáři adresar, tak se mi může stát, že

- zadám do prohlížeče adresu http://priklad.cz/adresar/**zadana-stranka.html**

- dostanu **obsah** z URL <http://priklad.cz/adresar/podstrcena-stranka.html>
- ale v prohlížeči stále **uvidím původní adresu** <http://priklad.cz/adresar/zadana-stranka.html>

1.14.2 Přesměrování místo podstrčení

Když v předchozím příkladu přidáme na konec řádku RewriteRule do hranatých závorek pravidlo [R=301], takto:

```
RewriteRule (.*) /vysledek.html [R=301]
```

tak se podstrčení změní na **přesměrování**.

- zadám třeba adresu <http://priklad.cz/adresar/dotaz.html>
- pak v řádku prohlížeče **uvidím jinou adresu**: <http://priklad.cz/adresar/vysledek.html>, protože mě server přesměrovává (a prohlížeč to akceptuje)
- a samozřejmě pak dostanu i obsah této stránky [vysledek.html](#)

Čili v tuto chvíli jde o přesměrování, nikoliv o podstrčení. Kdyby tam bylo jenom [R] bez =301, tak to taky bude fungovat, ale server vrátí kód odpovědi 302 found (což je většinou to samé, co 301, akorát 302 se bere jako dočasné přesměrování, 301 jako trvalé).

1.14.3 Kdy se podstrkává a kdy přesměrovává

Výchozí chování serveru je podstrkávání. V jakých případech probíhá **přesměrování**:

- existuje jasná instrukce, že se má přesměrovávat (např. to [R])
- nebo nelze podstrkávat -- jde o případy, kdy **nová adresa** začíná na <http://>. Pak server nedovolí stránku podstrčit, i kdyby byla z jeho serveru. Následující zápis přesměrovává, i když nemá to [R]:

```
RewriteRule (.* http://www.jakpsatweb.cz
```

1.15 Příklad jednoduchého regulárního výrazu

Mod_rewrite umožňuje využívat regulární výrazy.

Pravidlo, které **všechno** z aktuálního adresáře (ve kterém je .htaccess) **přesměruje** na hlavní stránku www.jakpsatweb.cz:

```
RewriteEngine on
RewriteRule (.* http://www.jakpsatweb.cz [R]
```

Ten zápis **(.*)** znamená "**cokoliv**" (přesněji řečeno libovoný počet (to je hvězdička) libovolných znaků (to je ta tečka)).

Pokud v tomto případě uživatel napíše **libovolnou adresu**, která míří do aktuálního adresáře (to je adresář, ve kterém mám .htaccess), **přesměruje ho to na** <http://www.jakpsatweb.cz>. Tím eRkem v závorce opět říkáme, že se bude přesměrovávat.

1.16 Proměnné z regulárů

V zavolaném url lze něco najít a použít to na volání něčeho jiného.

To "něco" bude proměnná. Třeba najdu **číslo článku** a použiju ho na volání skrytého url. Následující příklad předpokládá, že stránky jsou napsané v php a jejich adresy mají normálně za otazníkem parametry. Články mají třeba toto url:

nějakýweb.cz/skript.php?id=234

Já bych ale chtěl na tuto stránku odkazovat a zapisovat adresou bez otazníku, například

nějakýweb.cz/clanek-234

Udělám to tak, že v pravidle pro mod_rewrite najdu číslo článku jako proměnnou (bude se jmenovat \$1) a tuto proměnnou použiju při definici toho, co se má podstrčit. Zápis pravidla vypadá tato:

```
RewriteRule ^clanek-(.*) skript.php?id=$1
```

Vysvětlení zápisu:

- uživatel si vyžádá url třeba **clanek-543**
- mod_rewrite to uvidí a všimne si, že to odpovídá regulárnímu výrazu **^clanek-(.*)**. Závorce **(.*)** totiž odpovídá libovolný počet znaků, a odpovídá mu tedy i řetězec **"543"**. Závorka se uloží do proměnné **\$1** (jednička, protože je to první závorka).
- mod_rewrite dále podstrkává uživateli obsah, který najde na adrese **skript.php?id=\$1**
- což nyní odpovídá adrese **skript.php?id=543**, protože **\$1** se rovná **543**
- (tuto složitou adresu s otazníkem a parametry uživatel vůbec neuvidí, jedná se o skryté url, i když je funkční)
- uživatel dále vidí na konci adresy v prohlížeči **clanek-543** (je to tedy podstrkávání)

1.17 Podmínky RewriteCond

Když ale reguláry nestačí, nastupují další možnosti **podmínkování, kdy se má manipulace provést**, a kdy ne.

1.17.1 Přesměrování na doménu s www

Požadavky na můj web mohou požadavky přicházet s hostname "jakpsatweb.cz" nebo "www.jakpsatweb.cz". Preferuji, aby lidé i vyhledávače chodili jenom na verzi s www, a tak je přesměrovávám.

Příklad: Všechny požadavky, co míří na **jakpsatweb.cz** bez www, přesměruju na verzi domény s **www**:

```
RewriteEngine on
RewriteCond %{HTTP_HOST} ^jakpsatweb\.cz [NC]
RewriteRule (.*?) http://www.jakpsatweb.cz/$1 [R=301,QSA,L]
```

S pravidlem **RewriteRule** se pracuje pouze v případě, že je splněna podmínka **RewriteCond**.

V tomto případě jsem například testoval proměnnou **%{HTTP_HOST}**, ve které je hostname (doménová část) vyžádaného url. Pokud hostname ("jakpsatweb.cz" nebo "www.jakpsatweb.cz") začíná (to je ta stříška) rovnou řetězcem "jakpsatweb.cz", tak se pravidlo provede.

A proto když někdo požádá o **http://jakpsatweb.cz/cokoliv**, je přesměrován na **http://www.jakpsatweb.cz/cokoliv**. Příznak QSA v hranatých závorkách ještě zařídí, že se do nové adresy přenesou i query-string (část adresy za otazníkem).

Výše uvedený zápis musí být v rootu domény, protože jinak by se to **\$1** vyhodnocovalo relativně k adresáři (jméno adresáře by se uřízlo, nevím proč). Toto asi nějak souvisí s RewriteBase (nevím).

To **[NC]** v RewriteCond znamená, že nezáleží na velikosti znaků. Tu normálně server rozlišuje, což v tuto chvíli nechci.

1.17.2 Přesměrování na doménu bez www

```
RewriteEngine on
RewriteCond %{HTTP_HOST} ^www.example\.com [NC]
RewriteRule (.*?) http://example.com/$1 [R=301,QSA]
```

Přepište example.com svou doménou. Dá se to napsat i obecněji, takže není nutno zadávat doménu:

1.17.3 Podobný příklad: přesměrování staré domény na novou

```
RewriteCond %{HTTP_HOST} ^stara-domena\.cz [NC]
RewriteRule ^(.*)$ http://www.nova-domena.cz/$1 [R=301,QSA,L]
```

1.17.4 Symbolický zápis RewriteCond

RewriteCond se symbolicky zapisuje:

```
RewriteCond testovaný-řetězec regulár
# pravidla RewriteRule
```

V testovaném řetězci se dají používat proměnné, a v regulárech zvláštní znaky. Např v příkladu:

```
RewriteCond %{HTTP_HOST} ^jakpsatweb\.cz
```

%{HTTP_HOST} je hostname, tedy **celé jméno domény**.

Stříška v příkladu znamená **začátek řetězce**.

Zpětné lomítko před tečkou říká, že se hledá **tečka**. Kdyby tam to zpětné lomítko nebylo, tak se v tomto případě nic nestane, protože v hostname na tomto místě těžko může být něco jiného než tečka. Proto ji v některých příkladech vynechávám.

Více podmínek se řetězí operátory **[OR]** a píšou se na více řádků. Například:

```
RewriteCond %{REMOTE_HOST} ^domena1.* [OR]
RewriteCond %{REMOTE_HOST} ^domena2.*
```

1.18 Příznaky v hranatých závorkách

V příkladech použití mod_rewrite se na konci pravidel často používají příznaky v hranatých závorkách. Anglicky se jim říká *flags*.

[L] Poslední pravidlo, nic už dál nepřepisuj

[QSA] Do přepsané / přesměrované adresy přidej za otazník vše, co je za otazníkem v původním požadavku (query string). Zkratka QSA znamená "query string append". Jestliže naopak chcete query string useknout, ukončete druhý parametr otazníkem.

[R] bude se přesměrovávat (s kódem 302), nikoli podstrkávat

[R=301] přesměrování půjde s http kódem 301

[F] nastavuje kód 403 - zakázáno. V kombinaci s dobrou RewriteCond umožňuje podmíněně zakázat některá URL.

[T=mime-typ] umožňuje výsledek poslat s jiným mime-typem. Použitelné zejména při přesměrování na binární soubory.

[NC] nezáleží na velikosti písmen (vhodné zejména do řádku RewriteCond).

Více příznaků v hranatých závorkách se v odděluje čárkou.

1.18.1 Nastavení .htaccess

Zdroj: <http://www.itnetwork.cz/objektovy-mvc-redakcni-system-v-php-htaccess-autoloader-kontroler>

Příklad výpis souborů ve složce na webu, budou tedy zvenku skryty, což my chceme. Přidáme řádek:

```
Options -Indexes
```

Bude tento soubor stažen (pokud existuje) a pokud ne, budeme přesměrováni na index. Kdybychom přesměrovali úplně vše (bez těch několika podmínek), nemohli bychom stahovat žádné soubory a vždy by se nám zobrazil index. Přesměrování jsme si ještě pojistili výčtem nejdůležitějších přípon, které se nebudou přesměrovávat. Všechny ostatní URL adresy směřují na index.

2 Composer



- Nástroj určený pro správu knihoven a jiných zdrojů (tzv. závislostí, *dependencies*) v PHP.
- Umožňuje uživateli/programátorovi deklarovat pro každý svůj PHP projekt které závislé knihovny chce využívat a ty za něj poté jednoduchým a sjednoceným způsobem spravuje.

To zahrnuje:

- stažení knihoven v případě, že neexistují
- zjišťování nových verzí knihoven a jejich případná aktualizace
- kontrola požadavků pro každou knihovnu
- řízení automatického nahrávání tříd spravovaných knihoven v PHP (class autoloading)
- Composer se dá ovládat z příkazové řádky
- je multiplatformní – k dispozici je pro operační systémy Windows, Linux a iOS.

Při nahrávání knihoven Composer zejména spolupracuje se servery [GitHub](https://github.com).com a packagist.org.

2.1 Soubory Composeru

Composer používá soubory:

`composer.json`

- Obsahuje použité knihovny.
- Tento soubor je editován uživatelem.
- Je možné do něj možné uložit název projektu, jeho stručný popis, licenci, seznam autorů a kontakty na ně.

composer.lock

- Obsahuje verze použitých knihoven, které mají být pro daný projekt použity. Tento soubor je generován.

vendor/

- Knihovny jsou implicitně instalovány do adresáře `vendor/` v daném projektu.
- V něm jsou uloženy v adresáři odpovídající jménu autora nebo organizace, která knihovnu vyvíjí, v podadresáři odpovídající jménu knihovny.

vendor/composer/

V podadresáři `vendor/composer/` je zčásti instalovaná, zčásti generovaná skupina PHP souborů určená pro autoloading PHP tříd.

vendor/composer/installed.json

- Obsahuje informace o projektem používaných knihovnách a přístupu k jejich třídám pro autoloading.

2.2 Příkazy z příkazové řádky

Nejčastější příkazy pro composer jsou:

```
composer install
```

Tento příkaz se spouští typicky po vytvoření `composer.json` pro první inicializaci utility. Příkaz stáhne a nainstaluje požadované knihovny, prošetří jejich požadavky, vygeneruje soubory pro autoloading.

```
composer update
```

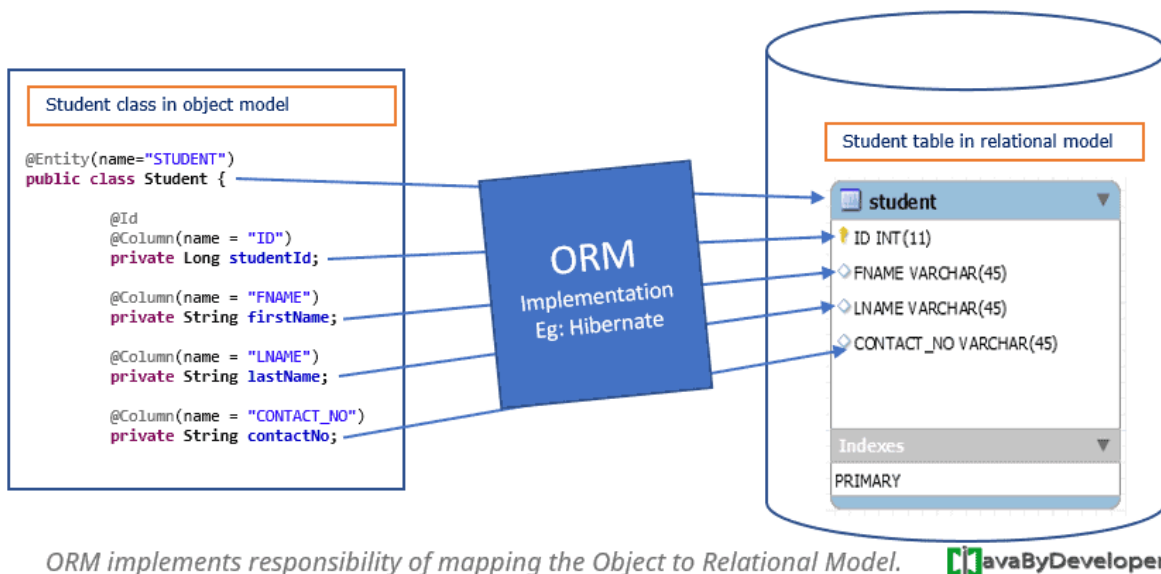
Tento příkaz projede seznam požadovaných knihoven a u těch, u kterých není určena konstantní verze, zjišťuje, neexistuje-li verze novější a v takovém případě se ji pokusí nainstalovat.

<https://getcomposer.org/>

3 ORM

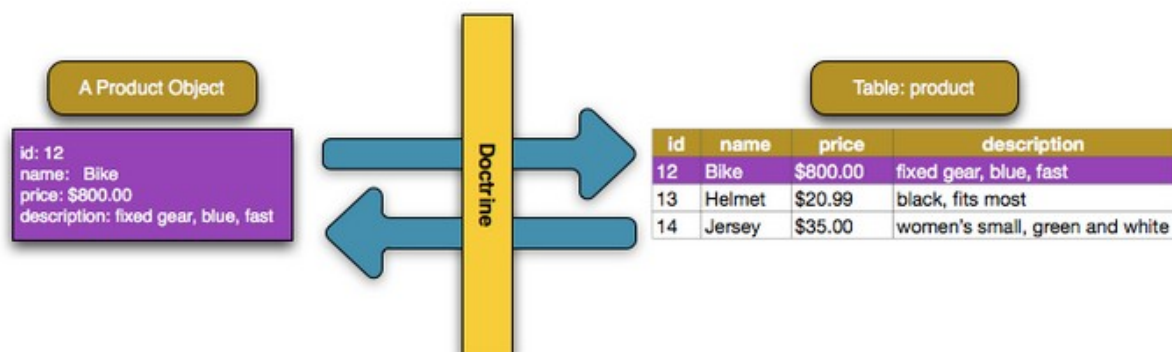
- Objektově relační zobrazení
- automatická konverze dat mezi relační databází a objektově orientovaným programovacím jazykem.
- Při modelování a vývoji aplikací je snaha co nejvěrněji zachytit realitu
- Objekty reálného světa jsou v aplikaci reprezentovány jako [entity](#). Využití ORM
- vývojář při práci s daty, které jsou v aplikaci reprezentovány právě pomocí objektů, do jisté míry odstíněn od nutnosti pracovat s [SQL](#) dotazy konkrétní relační databáze

hlavním cílem ORM je synchronizace mezi používanými objekty v aplikaci a jejich reprezentací v databázovém systému tak, aby byla zajištěna persistence dat. Vývojář potřebuje persistentně uchovávat objekty, ale nepotřebuje se starat, jak se tato persistence provede



Zdroj: <https://javabydeveloper.com/orm-object-relational-mapping/>

4 Doctrine (PHP)



Zdroj: <https://symfony.com/doc/current/doctrine.html>

- **Doctrine** je ORM framework pro jazyk PHP.
- Nabízí vysokou míru abstrakce databázové vrstvy použitím objektového přístupu.
- Umožňuje tak pracovat s daty jako s objekty.
- Jedna z klíčových vlastností Doctrine je psaní databázových dotazů použitím Doctrine Query Language (DQL),

4.1 Struktura frameworku

Framework Doctrine je rozdělen do tří vrstev.

- **Common**

Obsahuje obecná rozhraní, třídy a knihovny (práce s kolekcemi, anotacemi, cachování). Vrstva je na ostatních dvou vrstvách nezávislá, ale Common vrstvu ostatní vrstvy používají.

- **DBAL (DataBase Abstraction Layer)**

Zajišťuje databázovou nezávislost. Definuje DQL (Doctrine Query Language). Je závislá na vrstvě *Common*.

- **ORM (Object-Relational Mapping)**

Jedná se o nejvyšší vrstvu a zajišťuje mapování objektů do relační databáze, jejich ukládání a načítání.

4.2 Požadavky

Doctrine pro svůj běh vyžaduje minimálně PHP 5.3 (pro Doctrine 2). Podporuje tyto databáze: MySQL, PostgreSQL, Oracle a SQLite.

4.3 Definice a práce entitami

Entita je základní kámen Doctrine a vůbec objektového mapování. Reprezentuje objekt reálného světa. Zjednodušeně si lze entitu představit jako jeden řádek v databázové tabulce. Entitou může být např. student, učitel, předmět.

Příklad definice entity **Student**:

```
<?php

use Doctrine\ORM\Mapping as ORM;

/**
 * @ORM\Entity()
 * @ORM\Table(name="student")
 */
class Student
{
    /**
     * @var int
     * @ORM\Id()
     * @ORM\Column(type="integer")
     * @ORM\GeneratedValue()
     */
    private $id;

    /**
     * @var string
     * @ORM\Column(length=100)
     */
    private $jmeno;

    /**
     * @var string
     * @ORM\Column(length=100)
     */
    private $prijmeni;

    public function getId(): int
    {
        return $this->id;
    }

    public function getJmeno(): string
    {

```



```

        return $this->jmeno;
    }

    public function setJmeno(string $jmeno): void
    {
        $this->jmeno = $jmeno;
    }

    public function getPrijmeni(): string
    {
        return $this->prijmeni;
    }

    public function setPrijmeni(string $prijmeni): void
    {
        $this->prijmeni = $prijmeni;
    }
}

```

- Tři atributy: *id*, *jméno* a *příjmení*.
- Použití tzv. *komentářovou anotaci*. Druhý způsob definice entity vede přes XML nebo YAML soubory.
- Atribut *id* je primární klíč a jeho hodnota se automaticky generuje. Jako název tabulky a jejich sloupců se defaultně použijí názvy entit a jejich členských proměnných. Toto chování lze změnit uvedením atributu anotace *name*. S entitou se pak pracuje stejně jako s jakýmkoli jiným objektem.
- K ukládání nových entit a vyhledávání slouží třída **EntityManager**, jejíž instance je zavedena při startu aplikace. Příklad vytvoření nové entity a hledání:

```

<?php

$student = new Student;
$student->setJmeno('Franta');
$student->setPrijmeni('Vomáčka');

echo $student->getJmeno();
echo $student->getPrijmeni();

$entityManager->persist($student); // persistuje entitu a vytvoří ID
$entityManager->flush($student); // uloží fyzicky do DB

// vyhledání studenta s id=5
$student2 = $entityManager->find('Student', 5);

echo $student2->getJmeno();
echo $student2->getPrijmeni();

// změna a uložení
$student2->setJmeno('Jaromir');
$entityManager->flush(); // Uloží všechny změny

```

Příklad ukazuje základní vlastnosti objektového mapování, kdy s daty pracujeme jako s objektem a veškeré změny jsou přenášeny do databáze. Místo ukládání entit se v Doctrine používá výraz **persistence**. Důvod je spíš filozofický a vychází z významu slov, persistovat entitu můžeme jenom jednou a to po vytvoření nové instance.

Příklad smazání entity:

```
$entityManager->remove($student); // Odebere entitu v UnitOfWork  
$entityManager->flush(); // Proveďte fyzické odebrání v databázi
```

Metodě *remove* je předána instance entity, kterou chceme smazat.

5 DQL

- **DQL (Doctrine Query Language)** je dotazovací jazyk pro použití s frameworkem Doctrine.
- Je podobný jako SQL, ale vychází z jiných principů.
- Výsledkem DQL dotazu není jako v případě SQL tabulka s řádky, ale načtené instance entit.
- Při vytváření DQL dotazu vycházíme z definovaných názvů entit a jejich členských proměnných. Takto jsme zcela oproštěni od samotné existence databázových tabulek. Tím, že při definici entit jsme popsali vazby mezi nimi, výsledkem dotazu bude odkaz na asociaci, kterou jsou entity v dotazu propojeny.

S DQL je možno provádět operace *SELECT*, *UPDATE*, *DELETE*. Příkazy *GRANT*, *ALTER*, *CREATE*, *DROP* nelze provést, protože je to v rozporu se základní myšlenkou Doctrine (tyto příkazy se týkají přímo fyzické vrstvy).

Základní dotaz:

```
$query=$entityManager->createQuery("SELECT s FROM Project\Model\Student s WHERE  
s.jmeno = 'Franta'");
```

```
//pole objektů Student
```

```
$results=$query->getResult();
```

V tomto dotazu jsme použili alias pro entitu Student. Cestu k entitě definuje namespace entity.

Dotaz s využitím spojení tabulek:

```
$query = $entityManager->createQuery('SELECT a FROM Article a JOIN a.user u  
ORDER BY u.name ASC');  
// pole objektů Article  
$articles = $query->getResult();
```

Při spojování tabulek nemusíme uvádět, kterých sloupců se to týká, protože to framework rozpozná sám díky již nadefinovaným asociacím.

Dotaz využívající pojmenované parametry:

```
$query = $entityManager->createQuery('SELECT u FROM User u WHERE u.username  
= :name');  
$query->setParameter('name', 'Franta');  
$users = $query->getResult();
```

Kromě tohoto běžného použití DQL má programátor možnost vytvořit dotaz postupným voláním jeho metod a vytvořit tak specifický databázový dotaz. K tomu slouží

třída *Doctrine\ORM\QueryBuilder*. V konečné fázi je možno použít čisté SQL, ve všech třech případech dotazů (DQL, QueryBuilder, SQL) dojde k předání výsledné instance na entitu.

6 Framework

Framework (aplikační rámec) je softwarová struktura, která slouží jako podpora při programování a vývoji a organizaci jiných softwarových projektů. Může obsahovat podpůrné programy, knihovny API, podporu pro návrhové vzory.

Účel

Cílem frameworku je převzetí typických problémů dané oblasti, čímž se usnadní vývoj tak, aby se návrháři a vývojáři mohli soustředit pouze na své zadání. Například tým, který používá Apache Struts k vývoji webových stránek pro banku, se může zaměřit na to, jak se budou provádět bankovní operace, a ne jak zajistit bezchybnou navigaci mezi jednotlivými stránkami.

Vyskytují se námitky, že použitím frameworku bude kód pomalý či jinak neefektivní a že čas, který se ušetří použitím cizího kódu, se musí věnovat nastudování frameworku. Nicméně při jeho opakovaném nasazení nebo ve velkém projektu dojde k výrazné úspoře času. Při odinstalování frameworku již nebude možné některé aplikace spustit.

6.1 Architektura frameworku

Framework se skládá z tzv. *frozen spots* a *hot spots*.

Frozen spots definují celkovou architekturu softwarové struktury, její základní komponenty a vztahy mezi nimi. Tyto části se nemění při žádném použití frameworku.

Hot spots jsou komponenty, které spolu s kódem programátora vytvářejí zcela specifickou funkcionalitu, a proto jsou skoro pokaždé jiné.

V objektově orientovaném prostředí je framework tvořen abstraktními a klasickými (neabstraktními) třídami. *Frozen spots* pak mohou být reprezentovány abstraktními třídami a vlastní kód (*hot spots*) se přidá implementací abstraktních metod.

6.2 Příklady Frameworků

6.3 Laravel



Laravel, který byl představen v roce 2011, se stal nejpopulárnějším bezplatným open-source frameworkem PHP na světě. Proč? Protože dokáže bezpečně zpracovat složité webové aplikace, podstatně rychlejším tempem než jiné rámce. Laravel zjednodušuje proces vývoje usnadněním běžných úkolů, jako je směřování, relace, ukládání do mezipaměti a ověřování.

Důvody použití Laravelu

Laravel je vhodný při vývoji aplikací se složitými požadavky na back-end, ať už malými nebo velkými. Instalace Laravelu byla usnadněna zavedením Homestead, předem zabalené krabičky typu „vše v jednom“.

Je to rámec PHP plný funkcí, které vám pomohou přizpůsobit složité aplikace. Mezi ně patří: bezproblémová migrace dat, podpora architektury MVC, zabezpečení, směrování, modul šablon zobrazení a ověřování.

Laravel je vysoce expresivní a jeho rychlost a zabezpečení odpovídají očekávání moderní webové aplikace. Pro vývojáře, kteří chtějí vytvářet B2B nebo podnikové weby, které se budou vyvíjet s měnícími se webovými trendy, je Laravel cestou.

6.4 CodeIgniter



CodeIgniter, známý pro svou malou stopu (včetně dokumentace), má pouze velikost asi 2 MB, je rámec PHP vhodný pro vývoj dynamických webových stránek. Nabízí řadu předem připravených modulů, které pomáhají s konstrukcí robustních a opakovaně použitelných komponent.

Důvody použití CodeIgniter

CodeIgniter je lehký a přímý rámec PHP, který se na rozdíl od jiných rámců instaluje bezproblémově. Vzhledem k jednoduchému procesu instalace a vysoce ilustrované dokumentaci je ideální pro začátečníky.

Mezi klíčové funkce patří architektura MVC, špičkové zpracování chyb, vestavěné bezpečnostní nástroje a jednoduchá a vynikající dokumentace. Kromě toho vytváří škálovatelné aplikace.

Ve srovnání s jinými rámci je CodeIgniter podstatně rychlejší. Protože také nabízí solidní výkon, je to dobrá volba, pokud chcete vyvíjet odlehčené aplikace pro provoz na skromných serverech. Jedno upozornění: Vydání CodeIgniter jsou trochu nepravidelná, takže rámec není skvělou volbou pro aplikaci, která vyžaduje zabezpečení na vysoké úrovni.

6.5 Symfony



Rámec Symfony byl spuštěn v roce 2005 a přestože existuje mnohem déle než jiné rámce na tomto seznamu, je to spolehlivá a vyspělá platforma. Symfony je rozsáhlý rámec PHP MVC a jediný rámec, o kterém je známo, že dodržuje PHP a webové standardy na odpališti.

Důvody použití Symfony

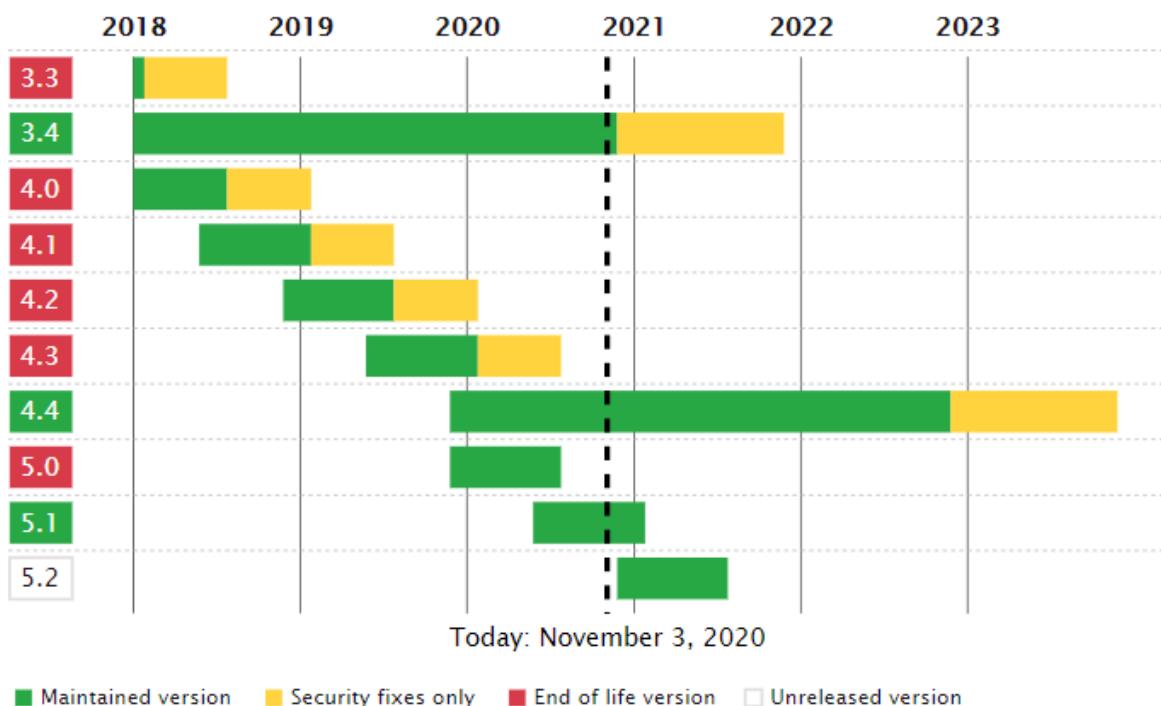
Symfony je ideální volbou pro vývoj velkých podnikových projektů. Instalace a konfigurace na většině platforem je snadná.

Jedna z jeho klíčových funkcí? Jedná se o opakovaně použitelné komponenty PHP. Může se také pochlubit nezávislostí na databázovém stroji a je stabilní, vyhovuje většině osvědčených postupů a návrhových vzorů pro web a umožňuje integraci s jinými knihovnami prodejců.

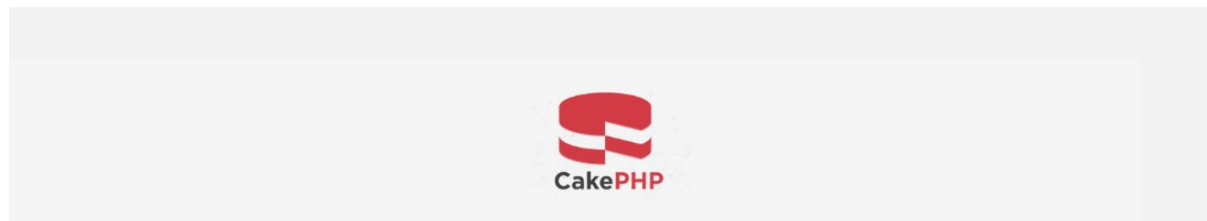
Symfony je také vysoce flexibilní a může se integrovat s většími projekty, jako je Drupal. Symfony a Laravel mají mnoho společných a jedinečných funkcí, takže je obtížné říci, který z těchto rámců je lepší.

Přestože se Laravel více zaměřuje na jednoduchost a poskytování hodnoty, a to i pro vývojáře, kteří nejsou pokročilí, Symfony se zaměřuje na pokročilé vývojáře a je o něco těžší začít. Kromě toho je bezpečnostní mechanismus Symfony trochu obtížný. A protože umožňuje vývojářům „dělat všechno“, může to být pomalejší než u jiných rámců.

Symfony Releases Calendar



6.6 CakePHP



Pokud hledáte sadu nástrojů, která je jednoduchá a elegantní, už nemusíte hledat. **CakePHP** vám pomůže vytvořit vizuálně působivé, funkcemi nabitě webové stránky. Kromě toho je CakePHP jedním z nejjednodušších rámců, které se lze naučit, zejména kvůli jeho CRUD (vytváření, čtení, aktualizace a mazání) rámci. CakePHP se dostal na trh počátkem roku 2000 a od té doby získal lepší výkon a mnoho nových komponent.

Důvody použití CakePHP

CakePHP se snadno a snadno instaluje, protože potřebujete pouze webový server a kopii rozhraní.

Je vhodnou volbou pro komerční aplikace díky bezpečnostním funkcím, které zahrnují prevenci vstřikování SQL, ověření vstupu, ochranu proti padělání žádostí mezi weby (CSRF) a ochranu mezi skripty mezi weby (XSS).

Mezi klíčové funkce patří moderní rámec, rychlé sestavení, správná dědičnost tříd, ověřování a zabezpečení. CakePHP navíc poskytuje skvělou dokumentaci, mnoho portálů podpory a prémiovou podporu prostřednictvím Cake Development Corporation.

6.7 Yii



Rámec [Yii](#) - což znamená Ano, to je! - je ve skutečnosti jednoduchý a evoluční. Jedná se o vysoce výkonný, na komponentách založený rámec PHP pro vývoj moderních webových aplikací. Yii je vhodný pro všechny druhy webových aplikací. Z tohoto důvodu se jedná o univerzální webový programovací rámec.

Důvody použití Yii

Yii má snadný proces instalace. Díky jeho robustním bezpečnostním funkcím je rámec vhodný pro vysoce bezpečné snahy, jako jsou projekty elektronického obchodování, portály, CMS, fóra a mnoho dalších.

Může se pochlubit vynikající rychlostí a výkonem, je vysoce rozšiřitelný a umožňuje vývojářům vyhnout se složitosti psaní opakujících se příkazů SQL, protože mohou modelovat data databáze z hlediska objektů.

Yii má základní vývojový tým a odborníky, kteří se podílejí na jeho vývoji. Díky rozsáhlé komunitě, kterou používáte, můžete odesílat problémy na fóra Yii a získat pomoc.

Yii je extrémně rozšiřitelný a můžete přizpůsobit téměř každou část kódu jádra. Pokud jej však používáte poprvé, buďte připraveni na strmou křivku učení.

6.8 Zend Framework



Rámec Zend je kompletní objektově orientovaná rámec, a skutečnost, že se používá k dispozici, jako je rozhraní a dědičnost umožňuje prodloužit. Byl postaven na agilní metodice, která vám pomůže dodávat vysoce kvalitní aplikace podnikovým klientům. Zend je vysoce přizpůsobitelný a dodržuje osvědčené postupy PHP - důležitý bod pro vývojáře, kteří chtějí přidat funkce specifické pro projekt.

6.8.1 Důvody pro použití Zendu

Zend framework se výborně hodí pro složité projekty na podnikové úrovni. Je to preferovaný rámec pro velká IT oddělení a banky.

Mezi klíčové funkce patří komponenty MVC, jednoduché cloudové API, šifrování dat a správa relací.

Může se integrovat s externími knihovnami a můžete použít pouze požadované komponenty. Rámec Zend přichází s extrémně dobrou dokumentací a má velkou komunitní základnu. Pokud jste však tvůrcem mobilních aplikací, připravte se na strmou křivku učení.

6.9 Phalcon



[Phalcon](#) , [full-stackový](#) framework PHP, který využívá návrhový vzor webové architektury MVC, byl původně napsán v C a C ++ a vydán v roce 2012. Jelikož je dodáván jako C-rozšíření, nemusíte se starat o učení programovacího jazyka C. .

6.9.1 Důvody použití Phalconu

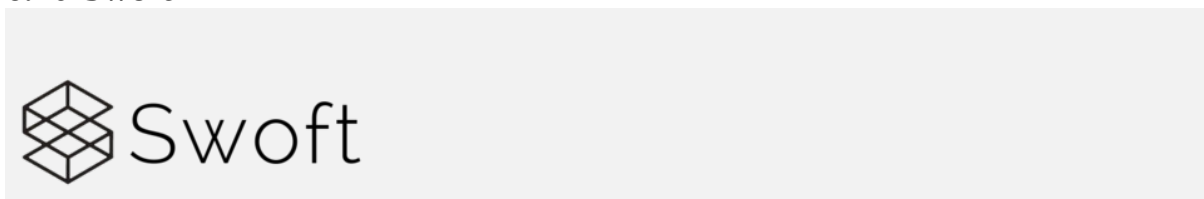
Phalcon se snadno instaluje a je vhodný pro vytváření vysoce konfigurovatelných webových aplikací, které jsou v souladu s pokyny pro vývoj podniku.

Mezi klíčové funkce patří zvýšená rychlost provádění, správa majetku, univerzální automatický podavač a špičkové zabezpečení a ukládání do mezipaměti.

Na rozdíl od jiných frameworků Phalcon optimalizuje výkon díky efektivnímu využití paměti. Pokud hledáte vytvořit bleskově rychlý web, vyzkoušejte Phalcon.

Negativní stránkou je, že vývojáři Phalconu opravují chyby trochu pomalu, což při dnešní potřebě vysoké úrovně zabezpečení nemusí fungovat.

6.10 Swift



Swift je vysoce výkonný rámec pro **mikroslužby coroutine** v PHP. Vychází již mnoho let a stala se nejlepším volbou pro php. Může to být jako Go, vestavěný webový server coroutine a běžný klient coroutine a je rezidentem v paměti, nezávisle na tradičním PHP-FPM. Existují podobné operace jazyka Go, podobné pružným anotacím rámce Spring Cloud.

6.10.1 Důvody použití Swift

Na základě Swoole native Coroutine přichází s rezidentní pamětí a balíčkem dalších funkcí Swoole.

Dodává se s efektivním fondem připojení Mysql / Redis / Rpc a opětovným připojením k odpojení všech připojení. Vývojáři se nestarají o sdružování připojení a byly implementovány odpovídající komponenty.

AOP lze použít pro všechny objekty spravované kontejnerem rozhraní. Použití AOP vám umožňuje řídit chování instančních objektů bez změny vnitřku instance.

Služba RPC je rozdělena na RPC Server a RPC Client a rámec poskytuje elegantnější způsob použití služeb RPC, jako je Dubbo.

S rámci sítě služeb, jako je Istio / Envoy, a poskytuje sadu komponent rychlého sestavení správy mikroslužeb pro malé a střední podniky, včetně registrace a zjišťování služeb, bucků služeb, omezování služeb a konfiguračních center.

6.11 PHPixie



Představený v roce 2012 a stejně jako FuelPHP, PHPixie realizuje návrhový vzor HMVC. Jeho cílem bylo vytvořit vysoce výkonný rámec pro weby jen pro čtení.

6.11.1 Důvody použití PHPixie

Je snadné začít s PHPixie, který je vhodný pro weby sociálních sítí, přizpůsobené webové aplikace a vývojové služby webových aplikací.

Mezi klíčové funkce patří architektura HMVC, standardní ORM (objektově-relační mapování), ověřování vstupů, možnosti autorizace, autentizace a ukládání do mezipaměti.

PHPixie je postaven pomocí nezávislých komponent. Z tohoto důvodu jej můžete použít bez samotného rámce. Všimněte si, že PHPixie má relativně málo modulů. Kromě toho postrádá podporu pro komponenty nezávisle vyrobené ze závislostí. Protože je relativně nový, je méně populární a má menší komunitu uživatelů než jiné rámce.

6.12 10. Slim



Slim Framework

Slim je další populární mikrorámec PHP, který pomáhá vývojářům rychle vytvářet jednoduché, ale výkonné webové aplikace a API.

Důvody použití Slim

Stejně jako PHPixie se Slim snadno naučí. Vývojáři PHP používají Slim k vývoji RESTful API a webových služeb. Mezi hlavní funkce patří směrování adres URL, relace a šifrování souborů cookie, ukládání do mezipaměti HTTP na straně klienta a další.

Je to nejlepší rámec pro malou webovou aplikaci, která nutně nevyžaduje rámec PHP s plným zásobníkem. Díky aktivní údržbě a přátelské dokumentaci je Slim super uživatelsky přívětivý.

Který framework PHP je pro vás vhodný?

Použití frameworků PHP zjednodušuje proces vývoje, což pomáhá minimalizovat pracovní zátěž. Každý rámec má své vlastní silné a slabé stránky a všechny se liší z hlediska komunity, dokumentace a databáze, kterou podporují.

6.13 Nette

český php Framework

Autor: David Grundl



šablonovací systém

ladící nástroje

efektivní databázovou vrstvu

důmyslné zabezpečení před zranitelnostmi

moderní framework s podporou HTML5, AJAX nebo SEO

s kvalitní dokumentací a nejaktivnější komunitou v ČR

s vyzrálým a čistým objektovým návrhem

vedoucím k dobrým návykům a dávajícím dostatek volnosti

Eliminuje bezpečnostní rizika. Můžete v něm tvořit e-shopsy, wiki, blogy, CMS, co jen vymyslíte.

Nette Framework používají významné společnosti jako třeba T-Systems, GE Money, Mladá fronta, VLTAVA-LABE-PRESS, Internet Info, DHL, Logio, ESET, Actum, Slevomat, Socialbakers, SUPRAPHON, pohání i stránky bývalého prezidenta Václava Klause. V anketě serveru Zdroják byl zvolen jako nejpoblárnější a nepoužívanější framework v České republice.

7 GitHub

- Verzovací systémy
- Webová služba podporující vývoj softwaru za pomoci verzovacího nástroje [Git](#). GitHub
- Bezplatný Webhosting pro open source projekty. Od 7. ledna 2019 je možné ukládat bezplatně i soukromé repositáře (dříve po zaplacení měsíčního poplatku).
- GitHub hostuje přes 100 milionů repositářů (od dubna 2019). Pro uživatele poskytuje funkce sociálních sítí – notifikace o změnách, diskuze nad kódem, návrhy změn, či zasílání vlastních řešení (pull-requesty).
- GitHub poskytuje další služby. Gist, jenž je součástí GitHubu, umožňuje verzování a rychlé sdílení kratších kódů.

7.1 Možnosti využití

GitHub se převážně využívá pro programování.

Kromě zdrojového kódu GitHub podporuje navíc zde uvedené formáty a vlastnosti:

- Dokumentace, včetně automaticky poskytovaných šablon typu ReadMe.md (čti mne), License.md (Licence) souborů v značkovacím jazyku (Markdown).
- Systém sledování problémů (Issue tracking), včetně požadavků na přidání dodatečných vlastností (tzv. features) s popisky, mezníky, zmocnění a s vyhledávači
- Vlastní řešení (Pull requesty) s revizí kódu a komentáři
- Historie verzování (Commit)
- Grafy: verzování, síť přispěvatelů a její členové, děrný štítek, frekvenčnost programování
- Integrace adresářů
- Hledání rozdílů (diff)
- Oznámení elektronickou poštou
- Možnost přihlášení někoho k oznámením a změnám tím, že ho zmíníme (@ mentioning)
- Emodži - možnost vložit
- Malé webové stránky mohou být hostovány z veřejných repozitářů na GitHubu. Formát URL je `http://uživatelské_jméno.github.io`
- Vnořené seznamy úkolů v souborech (time management)
- Vizualizace geoprostorových dat* 3D renderované soubory lze zobrazit pomocí nového integrovaného prohlížeče STL souborů, který zobrazí soubory na trojrozměrné plátno. Prohlížeč běží na WebGL a Three.js
- Nativní formát Photoshopu lze zobrazit a porovnat s přechozími verzemi stejného souboru
- Commit si můžete představit jako 1 kus práce

<https://www.kutac.cz/pocitace-a-internety/jak-na-git-dil-1> <https://www.kutac.cz/pocitace-a-internety/jak-na-git-dil-1>

The screenshot shows the GitHub interface for the repository 'smrkaf/udrzba'. At the top, there's a navigation bar with links to Pull requests, Issues, Marketplace, and Explore. Below this is a large banner with the text 'Learn Git and GitHub without any code!' and a 'Read the guide' button. The repository name 'smrkaf/udrzba' is displayed, along with statistics: 1 Watch, 0 Stars, and 0 Forks. The 'Code' tab is selected, showing a file tree with folders 'app', 'bin', 'src', and 'tests/AppBundle/Controller'. A yellow alert box at the top of the file list states: 'We found potential security vulnerabilities in your dependencies. Only the owner of this repository can see this message.' The right sidebar contains sections for 'About' (No description, website, or topics provided), 'Releases' (No releases published), and 'Packages'.

